

Diseño de la evaluación

1. Preguntas de investigación

1. ¿Cuál es la dificultad de cada zona para los jugadores?
 - a. ¿Qué zona es la más complicada para los jugadores?
 - b. ¿En cuál zona los jugadores suelen recibir daño más a menudo?
 - c. ¿En qué zona se suelen atascar los jugadores?
2. ¿Cuál es el enemigo más difícil para los jugadores?
3. ¿La curación que reciben los jugadores es adecuada?
4. ¿Cuál es la dificultad del nivel?

2. Métricas

Clarificaciones:

- Una zona se refiere al segmento de un checkpoint a otro o de el inicio hasta un checkpoint o desde un checkpoint hasta la meta
1. Distribución de muertes del jugador en cada Nivel **[4]**
 - a. Para todos los jugadores:
 - i. Sumar todas las muertes y hacer los porcentajes a partir de ahí
 2. Distribución de muertes del jugador en cada Zona de cada Nivel **[1]**
 - a. Usando un heatmap para representar donde ha muerto más
 3. Distribución de muertes del jugador según el tipo de obstáculo (enemigos, pinchos, etc.) **[1][2]**
 - a. Para todos los jugadores:
 - i. Sumar todas las muertes y hacer los porcentajes a partir de ahí
 4. Distribución de tiempo en el que el jugador tarda en pasarse cada zona. **[1]**
 - a. Media de los tiempos tardados en cada zona para todos los jugadores.
 5. Tasa de aciertos con respecto a disparos **[1][2]**
 - a. Media de las medias para todos los jugadores.
 6. Tasa de aciertos con respecto a ataques melee.
 - a. Media de las medias para todos los jugadores.
 7. Distribución de daño que recibe el jugador en cada Nivel **[1][2][4]**
 - a. Número de puntos de daño (no por porcentaje)
 - b. Media de el daño sufrido en cada nivel para todos los jugadores
 8. Distribución de daño que recibe el jugador según el tipo de obstáculo o enemigo **[1][2][4]**
 - a. Número de puntos de daño (no por porcentaje)
 9. Frecuencia con la que se cura el jugador sobre el tiempo **[3]**
 - a. Media de las frecuencias para todos los jugadores.
 10. Proporción de la curación recibida que es malgastada por sobrepasar la vida máxima **[3]**
 - a. Media de las proporciones para todos los jugadores.

3. Eventos por Telemetría

Clase padre y eventos generales

- TelemetryEvent: (clase padre)
 - **float** Timestamp (momento en el tiempo en el que se lanza el evento)
 - **int** SesionID
- SesionStart:
- SessionEnd:

Eventos específicos a nuestra evaluación

- playerDeath:
 - **int** levelID
 - **float** cordX
 - **float** cordY
 - **enum** deathCause {PengEnem, IceEnem, Spikes, etc.}
- playerShoot:
- enemyBulletHit:
- playerMelee:
- enemyMeleeHit:
- playerHit:
 - **float** cordX
 - **float** cordY
 - **enum** hitCause {PengEnem, IceEnem, etc.}
 - **float** currentHealth
 - **float** hitDamage
- playerHealing:
 - **float** previousHealth
 - **float** healingAmount
 - **float** finalHealth
- playerCheckPoint:
 - **int** levelID
 - **int** checkpointID

- playerEnd:
 - **Int** levelID
 - **float** currentHealth

4. Utilización de Eventos

Para la métrica 1 [Distribución de muertes del jugador en cada Nivel] vamos a utilizar:

- **PlayerDeath**: Evento que se lanzará cada vez que se muera el jugador. Proporcionará las coordenadas **CordX**, **CordY** para guardar la posición y **LevelID** para guardar el nivel en el que murió.

Para la métrica 2 [Distribución de muertes del jugador en cada Zona de cada Nivel] vamos a utilizar:

- **PlayerDeath**: Evento que se lanzará cada vez que se muera el jugador. Proporcionará las coordenadas **CordX**, **CordY** para guardar la posición

Para la métrica 3 - Distribución de muertes del jugador según el tipo de obstáculo

- **PlayerDeath**: Evento que se lanzará cada vez que se muera el jugador. Proporcionará las coordenadas **CordX**, **CordY** para guardar la posición, y el **DeathCause** para guardar qué obstáculo es el que ha matado al jugador.

Para la métrica 4 - Distribución de tiempo en el que el jugador tarda en pasarse cada zona vamos a utilizar:

- **PlayerCheckpoint**: Evento que se lanza cada vez que llegamos a un checkpoint. Proporciona el **CheckpointID** y **LevelID** para guardar el checkpoint y el nivel al que ha llegado el jugador. En la clase padre usamos el **Timestamp** para saber en qué momento el jugador ha alcanzado el checkpoint.
- **PlayerEnd**: Se lanza cada vez que llegamos al final de un nivel. Proporciona el **LevelID** para saber cuándo se pasa de nivel. En la clase padre usamos el **Timestamp** para saber en qué momento el jugador ha alcanzado el checkpoint.

Para la métrica 5 - Tasa de aciertos con respecto a disparos vamos a utilizar los eventos:

- **PlayerShoot**: Evento que se lanza cada vez que el jugador dispara (sumando +1 el contador de disparos totales en un nivel).
- **EnemyBulletHit**: Evento que se lanza cada vez que el disparo del jugador acierta un enemigo (sumando +1 el contador de disparos acertados durante el nivel). Luego se podrá dividir el contador anterior entre éste para obtener la tasa de aciertos.

Para la métrica 6 - Tasa de aciertos con respecto a ataques melee vamos a utilizar los eventos:

- **PlayerMelee**: Evento que se lanza cada vez que el jugador usa el ataque a melee (sumando +1 el contador de ataques totales en un nivel).
- **PlayerMeleeHit**: Evento que se lanza cada vez que el jugador acierta un ataque a melee (sumando +1 el contador de ataques acertados en un nivel). Luego se podrá dividir el contador anterior entre éste para obtener la tasa de aciertos.

Para la métrica 7 - Distribución de daño que recibe el jugador en cada Nivel vamos a utilizar los eventos:

- **PlayerHit:** Evento que se lanza cada vez que el jugador es golpeado por un enemigo u obstáculo que le quite vida. Proporciona un enumerado HitCause para saber qué es lo que ha hecho daño al jugador. También posee un float CurrentHealth y float HitDamage con el que calculamos la distribución del daño a través de todo el nivel.

Para la métrica 8 - Distribución de daño que recibe el jugador según el tipo de obstáculo o enemigo

- **PlayerHit:** Evento que se lanza cada vez que el jugador es golpeado por un enemigo u obstáculo que le quite vida. Proporciona un enumerado HitCause para conocer el obstáculo o enemigo que ha causado el daño.

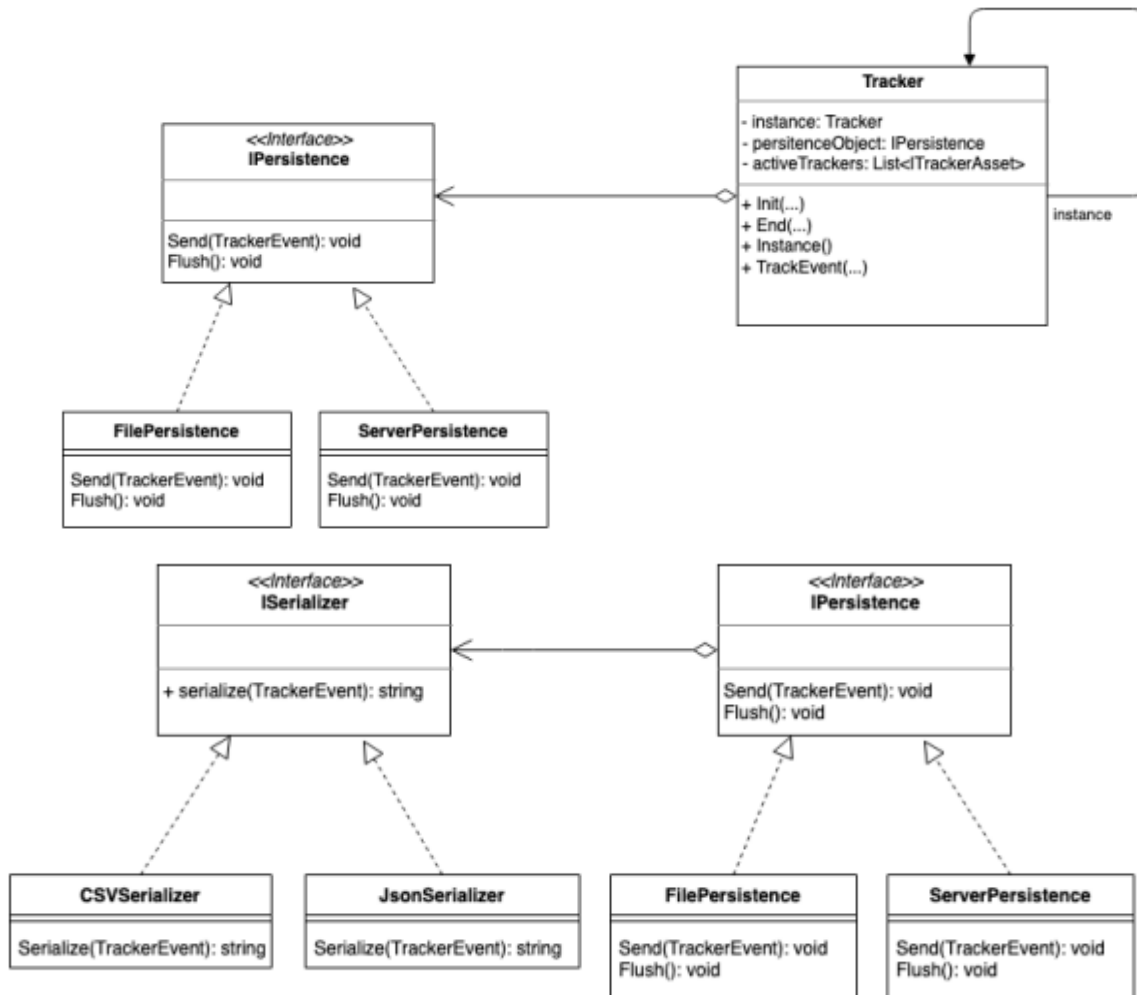
Para la métrica 9 - Frecuencia con la que se cura el jugador sobre el tiempo vamos a utilizar:

- **PlayerHealing:** Evento que se lanza cada vez que el jugador usa un ítem de cura.

Para la métrica 10 - Proporción de la curación recibida que es malgastada vamos a utilizar:

- **PlayerHealing:** Evento que se lanza cada vez que el jugador usa un ítem de cura, proporciona la salud anterior (PreviousHealth) y posterior (FinalHealth). Su diferencia representa la cura real, que se resta a la salud teóricamente curada (HealAmount) para saber la curación que no se ha podido usar.

5. Arquitectura



6. Cambios en clases del código original para la instrumentalización:

- **GameManager**: Añadido un método con nombre "CheckTelemetrySys", que lanza un evento "PlayerEnd" cuando el jugador llega al final del nivel, a la vez que llama al `Flush()` del **Tracker** para guardar datos en disco.
- **SaveManager**: Añadido parámetro que guarda el id del último checkpoint tocado: `trackerCP_ID`.
- **MightyLifeComponent**: Añadido un método con nombre "CheckTelemetrySysHealing", que lanza un evento "PlayerHealing" cuando el jugador obtiene una cura. Añadido método "CheckTelemetrySysDeath" que lanza "PlayerDeath" cuando el jugador muere por pinchos o lava. Añadido método "CheckTelemetrySysLanzallamas" que lanza "PlayerDeath" cuando el jugador muere

por tocar fuego de un lanzallamas estando a baja vida, o lanza "PlayerHit" cuando el jugador toca el fuego pero todavía le queda vida.

Añadido método "CheckTelemetrySysCheckpoint" que lanza un evento "PlayerCheckpoint" cuando el jugador toca un nuevo checkpoint, a la vez que guarda el ID del checkpoint nuevo en SaveManager y que llama al Flush() del Tracker para guardar datos en disco.

- EnemyHealth: Añadido método "CheckTelemetrySys" que lanza un evento "PlayerDeath" cuando el enemigo logra matar al jugador tras quitarle lo que le quedaba de vida, o lanza "PlayerHit" cuando el enemigo golpea al jugador, pero todavía le queda vida a este último.
- CarambanosDeHielo: Añadido método "CheckTelemetrySys" que lanza un evento "PlayerDeath" cuando el carámbano de hielo logra matar al jugador tras quitarle lo que le quedaba de vida, o lanza "PlayerHit" cuando el carámbano de hielo golpea al jugador, pero todavía le queda vida.
- BulletCollisionComponent: Añadido método "CheckTelemetrySys" que lanza un evento "PlayerDeath" cuando una bala enemiga logra matar al jugador tras quitarle lo que le quedaba de vida, o lanza "PlayerHit" cuando la bala enemiga impacta contra el jugador, pero todavía le queda vida. También se ha modificado la clase para que cuando una bala del jugador impacte contra un enemigo, envíe un evento "EnemyBulletHit".
- ShootingComponent: Se ha modificado la clase para que cuando el jugador dispare una bala, se lance un evento "PlayerShoot".
- MeleeComponent: Se ha modificado la clase para que cuando el jugador ataque con la espada melee, se lance un evento "PlayerMelee". Si el ataque acierta sobre algún enemigo, se lanza un evento "EnemyMeleeHit".
- IntroMenuCommands: Se ha modificado la clase para que cuando se llame a SalirR(), antes del Application.Quit() se llama también al End() del Tracker para lanzar un evento "SesionEnd".

En todos los lanzamientos de eventos al Tracker, si este no tenía una instancia, se llama al Init() del Tracker que lanza un evento "SesionStart".